Claude Code in Action

Complete Course Reference & Detailed Explanation

21 Lessons | 100% Complete | Certificate Earned

1. Course Overview

Claude Code in Action is a structured, hands-on course designed for developers who want to integrate AI-assisted coding into their professional workflow. The course covers everything from initial setup to advanced automation, walking learners through practical, real-world scenarios using Anthropic's Claude Code tool.

By the end of this course, you will understand how Claude Code works at a fundamental level, how to configure it for your specific projects, how to extend its capabilities using the Model Context Protocol (MCP) and custom commands, and how to automate it programmatically via hooks and the Claude Code SDK.

2. What is Claude Code?

This section introduces Claude Code, provides foundational context about AI-powered coding assistants, and demonstrates the tool in a live coding scenario.

2.1 Introduction

Claude Code is a terminal-based AI coding assistant built by Anthropic on top of the Claude language model. Unlike browser-based AI tools, Claude Code runs directly in your development environment, giving it access to your local filesystem, codebase, and shell. It understands your project structure, can read and write files, execute commands, and reason about multi-file changes.

- It is designed to be used in real development workflows, not just for simple code generation.
- Claude Code operates as an agentic tool: it can plan, execute, and iterate on tasks autonomously.
- It is built on the same underlying model as Claude.ai but is optimized for developer use cases.

2.2 What is a Coding Assistant?

A coding assistant is an AI system that helps developers write, understand, debug, and improve code. Unlike a traditional autocomplete tool (like early Copilot versions), modern coding assistants can:

- Understand natural language instructions and convert them into code changes.

- Read existing code and reason about its structure, logic, and intent.
- Suggest architectural improvements, find bugs, and explain complex logic.
- Execute multi-step tasks like refactoring a module, writing tests, or migrating an API.

Claude Code sits at the more powerful end of this spectrum. It acts less like an autocomplete engine and more like a capable junior engineer you can delegate tasks to.

2.3 Claude Code in Action

This lesson is a live demonstration showing Claude Code solving a real development task from start to finish. The instructor walks through an end-to-end example including:

- Opening a project in the terminal.
- Asking Claude Code to understand the existing codebase.
- Issuing a natural language instruction to make a specific code change.
- Reviewing the changes Claude Code proposes and accepting or rejecting them.

This lesson gives learners a concrete mental model of how the tool behaves before they set it up themselves.

3. Getting Hands On

This section takes you from zero to a working Claude Code setup. You will install the tool, configure a project, learn how to feed it useful context, and make your first AI-assisted code change.

3.1 Claude Code Setup

Installation and initial configuration of Claude Code is covered in detail. Key topics include:

- Prerequisites: Node.js, npm, and an Anthropic API key.
- Installing Claude Code globally via npm.
- Authenticating with your Anthropic account.
- Running Claude Code for the first time and understanding the terminal interface.
- Setting up your environment so Claude Code has appropriate permissions.

The lesson also covers common installation issues and how to resolve them on Mac, Windows, and Linux.

3.2 Project Setup

Not all projects are created equal when it comes to AI assistance. This lesson explains how to structure a project so Claude Code can understand it quickly and work effectively. Topics covered:

- The importance of a clear directory structure.
- How Claude Code reads your project at startup.
- What kinds of files Claude Code needs access to and which to exclude.
- Using .gitignore and similar files to keep noise out of the context window.
- Best practices for naming conventions and file organization.

3.3 Adding Context

Claude Code's quality of output is directly tied to the quality of context it receives. This lesson covers the different mechanisms for providing context:

- CLAUDE.md files: a special markdown file at the root of your project that Claude Code reads automatically. Use it to describe the project, its architecture, coding conventions, and any other information Claude should always know.
- Inline file references: mentioning specific files in your prompt so Claude reads them.
- System-level context: environment variables and configuration that inform Claude's behavior.
- How to think about the context window and avoid overloading it with irrelevant information.

3.4 Making Changes

With setup and context in place, this lesson walks through making actual code changes with Claude Code. You will learn:

- How to phrase instructions clearly and effectively.
- The diff review workflow: Claude proposes changes, you review and approve.
- How to iterate: refining instructions when Claude's first attempt is not quite right.
- Undoing changes and recovering if Claude makes a mistake.
- When to let Claude work autonomously versus when to stay in the loop.

4. Controlling Context

Advanced context management separates power users from casual users. This section covers custom commands, MCP server integration, and GitHub connectivity.

4.1 Custom Commands

Custom commands allow you to define reusable, parameterized prompts that can be triggered with a slash command inside Claude Code. This is useful for repetitive tasks like running a specific type of code review, generating boilerplate, or performing a standard refactoring.

- How to define custom commands in your project or globally.
- The command syntax and how to include dynamic parameters.
- Examples: /review, /test, /document, /migrate.
- How custom commands reduce prompt engineering overhead for your team.

Custom commands are stored in a commands directory and are versioned with your project, making them shareable across a team.

4.2 MCP Servers with Claude Code

The Model Context Protocol (MCP) is an open standard that allows Claude Code to connect to external tools, APIs, and data sources. This dramatically expands what Claude Code can do beyond your local filesystem.

- What MCP is and how the client-server architecture works.
- How to configure MCP servers in your Claude Code settings.
- Examples of useful MCP servers: database access, web search, documentation lookup, Jira, GitHub, and more.
- Writing a simple custom MCP server to expose your own internal tools to Claude Code.

MCP transforms Claude Code from a local coding tool into a connected AI agent that can interact with your entire development ecosystem.

4.3 GitHub Integration

This lesson covers the native GitHub integration, which allows Claude Code to interact with repositories, pull requests, issues, and more directly from the terminal.

- Authenticating Claude Code with GitHub.
- Asking Claude to review a pull request or summarize recent commits.
- Creating branches, commits, and PRs via natural language.
- Using Claude Code in CI/CD pipelines via GitHub Actions.

5. Hooks and the SDK

Hooks and the SDK unlock the deepest level of customization, allowing you to intercept Claude Code's behavior programmatically and integrate it into automated workflows.

5.1 Introducing Hooks

Hooks are scripts or functions that run automatically at specific points in Claude Code's execution lifecycle. They allow you to:

- Enforce policies (e.g., always run linting before accepting a change).
- Log or audit every action Claude takes.
- Inject additional context automatically at the start of every session.
- Post-process Claude's output before it is written to disk.

Think of hooks as middleware for Claude Code. They give you fine-grained control without having to modify every prompt.

5.2 Defining Hooks

This lesson covers the technical structure of a hook definition. Key concepts:

- Hook types: pre-tool, post-tool, pre-session, post-session, and more.
- The hook configuration file format (JSON or YAML).
- How to specify which events trigger a hook.
- Passing data between Claude and the hook script.

5.3 Implementing a Hook

A step-by-step walkthrough of building a real hook from scratch. The example in this lesson implements a hook that automatically runs your test suite after every code change Claude makes, and feeds the test results back to Claude so it can fix any failures.

- Writing the hook script in Node.js or Python.
- Registering the hook in your configuration.
- Testing and debugging the hook.
- Handling errors and edge cases.

5.4 Gotchas Around Hooks

Common mistakes and pitfalls when working with hooks, and how to avoid them:

- Infinite loops: a hook that triggers Claude, which triggers the hook again.
- Performance: hooks that are too slow and block Claude's workflow.
- Scope issues: hooks that unintentionally affect unrelated sessions.
- Security: hooks that expose sensitive data or run unsafe commands.
- Debugging tips when hooks fail silently.

5.5 Useful Hooks!

A curated set of practical, production-ready hooks you can drop into your project immediately:

- Auto-lint hook: runs ESLint or Prettier after every file write.
- Test-on-change hook: runs relevant unit tests after each modification.
- Context injection hook: prepends your CLAUDE.md content to every prompt automatically.
- Change logger hook: records every file changed by Claude to an audit log.

5.6 Another Useful Hook

A second deep-dive hook example, this time implementing a security-scanning hook that runs a static analysis tool on any file Claude modifies and flags potential vulnerabilities before the change is accepted. This lesson reinforces the hook implementation pattern and demonstrates a real-world security use case.

5.7 The Claude Code SDK

The Claude Code SDK allows you to control Claude Code programmatically from your own Node.js or Python applications. This is useful for:

- Building custom development tools that leverage Claude Code internally.
- Automating large-scale code migrations or transformations.
- Integrating Claude Code into your CI/CD pipeline or internal developer platform.
- Creating batch processing workflows where Claude processes many files in sequence.

The SDK exposes the full power of Claude Code through a clean programmatic interface, with support for streaming responses, tool use, and session management.

6. Wrapping Up

6.1 Quiz on Claude Code

A comprehensive quiz testing your understanding of all major topics in the course, including: Claude Code setup, context management, custom commands, MCP, hooks, and the SDK. The quiz reinforces key concepts and helps identify any gaps in understanding before moving to production use.

6.2 Summary and Next Steps

A recap of everything covered in the course, distilled into actionable takeaways:

- Claude Code is most powerful when given rich, relevant context via CLAUDE.md and project structure.
- Custom commands and MCP servers let you tailor Claude Code to your team's exact workflow.
- Hooks provide programmatic control over Claude's behavior at every step of its lifecycle.
- The SDK is the gateway to fully automated, production-grade AI development pipelines.

Recommended next steps after completing the course:

- Set up Claude Code on your current project and write a CLAUDE.md.
- Identify two or three repetitive tasks and convert them to custom commands.
- Explore the MCP ecosystem and connect one external tool relevant to your stack.
- Implement a simple lint or test hook to start automating quality checks.

7. Full Curriculum at a Glance

#	Lesson Title	What You Will Learn
1	**Course Overview**	High-level introduction to the course structure, goals, and what you will build.
2	**Introduction**	What Claude Code is, how it differs from other AI tools, and its core design philosophy.
3	**What is a coding assistant?**	Deep dive into the category of AI coding assistants and where Claude Code fits.
4	**Claude Code in action**	Live demonstration of Claude Code solving a real coding task end to end.
5	**Claude Code setup**	Installation, authentication, and first-run configuration across all major operating systems.
6	**Project setup**	How to structure your project for optimal Claude Code performance.
7	**Adding context**	CLAUDE.md, inline references, and strategies for effective context management.
8	**Making changes**	The full workflow: prompting, reviewing diffs, iterating, and undoing changes.
9	**Course satisfaction survey**	Share feedback to help improve the course.
10	**Custom commands**	Define and use reusable slash commands for repeated tasks.

#	Lesson Title	What You Will Learn
11	**MCP servers with Claude Code**	Connect external tools and data sources via the Model Context Protocol.
12	**GitHub integration**	Link Claude Code to GitHub for PR reviews, commits, and CI/CD workflows.
13	**Introducing hooks**	What hooks are, why they matter, and their place in the execution lifecycle.
14	**Defining hooks**	Hook configuration syntax, event types, and data passing.
15	**Implementing a hook**	Step-by-step: build a test-runner hook that feeds results back to Claude.
16	**Gotchas around hooks**	Infinite loops, performance traps, scoping, security, and debugging tips.
17	**Useful hooks!**	Four production-ready hooks: auto-lint, test-on-change, context injection, audit log.
18	**Another useful hook**	Security scanning hook: runs static analysis and flags vulnerabilities automatically.
19	**The Claude Code SDK**	Programmatic control of Claude Code via Node.js/Python for automation and tooling.
20	**Quiz on Claude Code**	Comprehensive knowledge check across all course topics.
21	**Summary and next steps**	Key takeaways, recommended actions, and further learning resources.

8. Key Concepts Glossary

Term	Definition
Claude Code	Terminal-based AI coding assistant by Anthropic, built on the Claude model.
CLAUDE.md	A special markdown file that Claude Code reads automatically to understand your project.
Context Window	The amount of text Claude can process in a single interaction. Managing it well is critical for quality output.
MCP (Model Context Protocol)	An open standard for connecting Claude Code to external tools, APIs, and data sources.
MCP Server	A lightweight service that exposes external capabilities (databases, APIs, tools) to Claude via MCP.

Custom Command	A reusable, parameterized prompt triggered by a slash command (e.g., /review).
Hook	A script that runs automatically at a specific point in Claude Code's lifecycle (e.g., after a file is written).
Claude Code SDK	A Node.js/Python library for controlling Claude Code programmatically from your own applications.
Agentic AI	An AI system capable of planning and executing multi-step tasks autonomously with minimal human intervention.
Diff Review	The workflow where Claude proposes file changes as a diff that you review before they are applied.

9. Resources & Links

- **https://docs.anthropic.com/claude/docs/claude-code:** Official Docs
- **https://console.anthropic.com:** Anthropic Console
- **https://modelcontextprotocol.io:** Model Context Protocol
- **https://github.com/anthropics/claude-code:** Claude Code GitHub
- **https://community.anthropic.com:** Anthropic Community Forum